Fundusze Europejskie Wiedza Edukacja Rozwój
Politechnika Wrocławska
Unia Europejska Europejski Fundusz Społeczny

„ZPR PWr – Zintegrowany Program Rozwoju Politechniki Wrocławskiej”

Lecturer: dr hab. inż. Bogdan Trawiński, prof. uczelni

IT Project Management Support

Lecture 4

Requirement Management

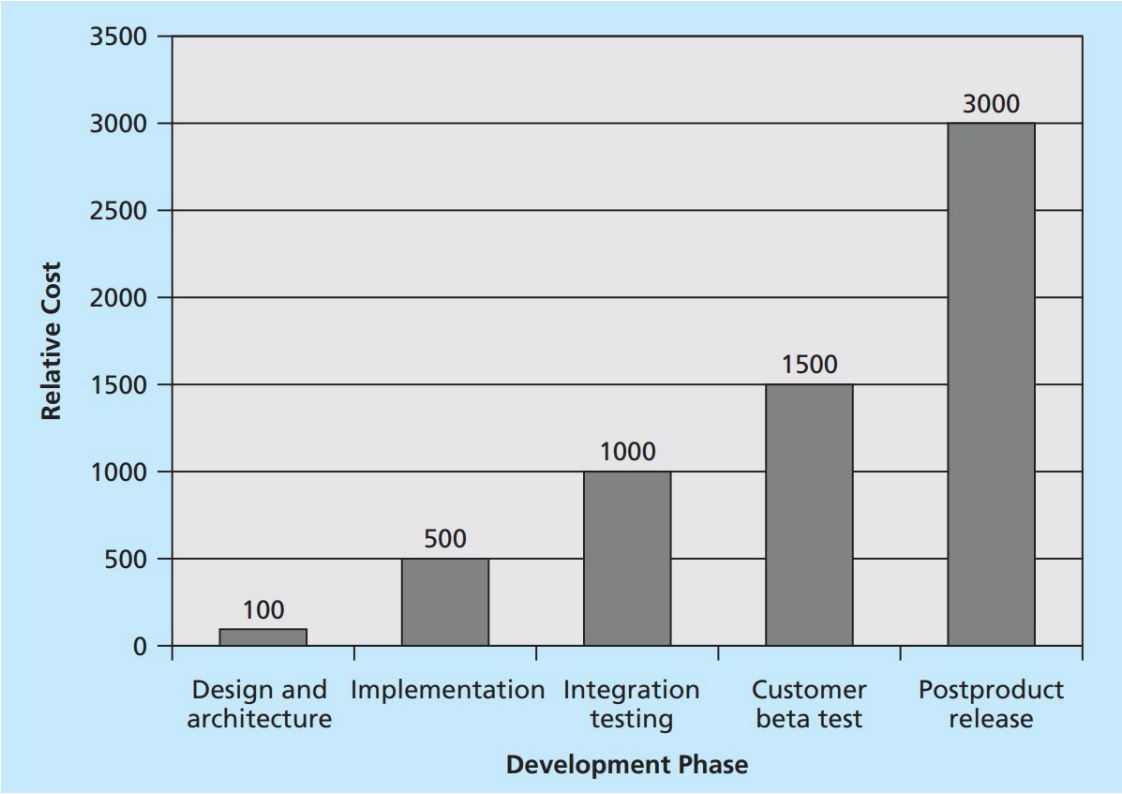
Project scope management

- A critically important and difficult aspect of project management is defining the scope of a project.
- Scope refers to all the work involved in creating the products of the project and the processes used to create them.
- Project scope management includes the processes involved in defining and controlling what work is or is not included in a project.
- It ensures that the project team and stakeholders have the same understanding of what products the project will produce and what processes the project team will use to produce them.
- Six main processes are involved in project scope management according to the PMBOK Guide:

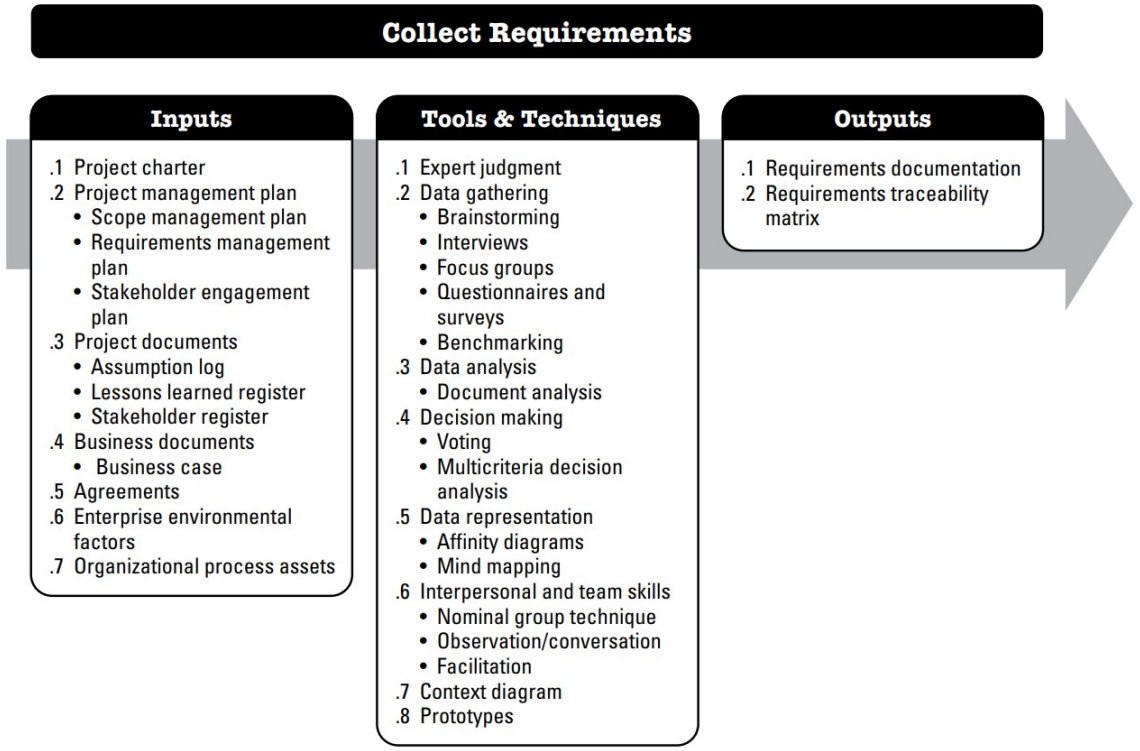
1. **Planning scope management** involves determining how the project's scope and requirements will be managed. The project team works with appropriate stake-holders to create a scope management plan and requirements management plan.
2. **Collecting requirements** involves defining and documenting the features and functions of the products as well as the processes used for creating them. The project team creates requirements documentation and a requirements traceability matrix as outputs of the requirements collection process
3. **Defining scope** involves reviewing the scope management plan, project charter, requirements documents, and organizational process assets to create a scope statement, adding more information as requirements are developed and change requests are approved. Outputs of scope definition are the project scope statement and updates to project documents.

4. **Creating the WBS** involves subdividing the major project deliverables into smaller, more manageable components. Outputs include a scope baseline (which includes a WBS and a WBS dictionary) and updates to project documents.
5. **Validating scope** involves formalizing acceptance of the project deliverables. Key project stakeholders, such as the customer and sponsor for the project, inspect and then formally accept the deliverables during this process. The outputs of this process are accepted deliverables, change requests, work performance information, and updates to project documents.
6. **Controlling scope** involves controlling changes to project scope throughout the life of the project. Scope changes often influence the team’s ability to meet project time and cost goals, so project managers must carefully weigh the costs and benefits of scope changes. The outputs of this process are work performance information, change requests, and updates to the project management plan, project documents, and organizational process assets.

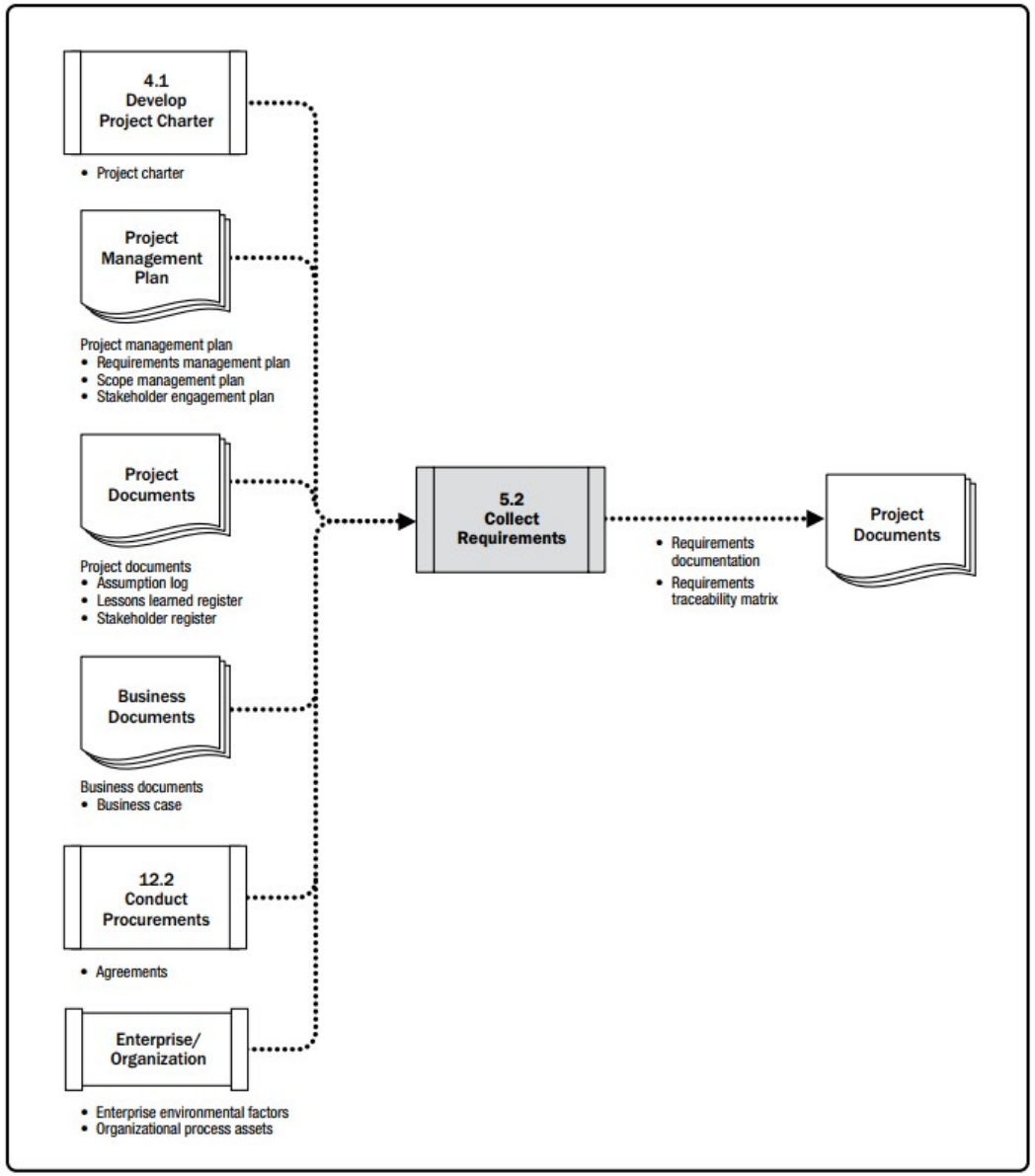
- The second step in project scope management is often the most difficult: collecting requirements.
- A major consequence of not defining requirements well is rework, which can consume up to half of project costs, especially for software development projects.
- As illustrated in figure below, it costs much more (up to 30 times more) to correct a software defect in later development phases than to fix it in the requirements phase.



- Collecting Requirements is the process of determining, documenting, and managing stakeholder needs and requirements to meet objectives.
- The key benefit of this process is that it provides the basis for defining the product scope and project scope.



Collect Requirements: Inputs, Tools & Techniques, and Outputs taken from the PMBOK Guide, ch. V, page 138



Collect Requirements: Data Flow Diagram taken from the PMBOK Guide, ch. V, page 139

What is Software Requirement?

Requirement is a condition or capability possessed by the software or system component in order to solve a real world problem.

The problems can be to automate a part of a system, to correct shortcomings of an existing system, to control a device, and so on.

IEEE defines requirement as:

1) A condition or capability needed by a user to solve a problem or achieve an objective.
2) A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed documents.
3) A documented representation of a condition or capability as in (1) or (2).

What is Software Requirement?

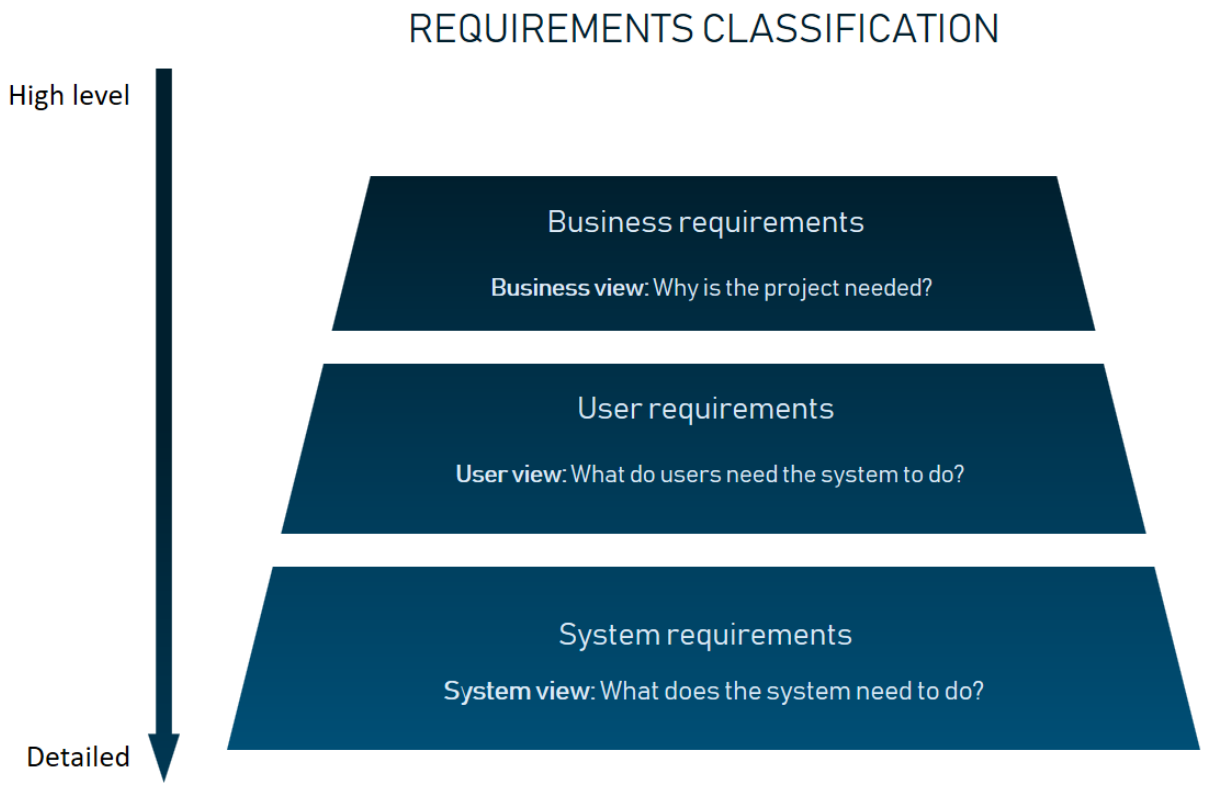
Requirement is a condition or capability possessed by the software or system component in order to solve a real world problem.

The problems can be to automate a part of a system, to correct shortcomings of an existing system, to control a device, and so on.

IEEE defines requirement as:

1) A condition or capability needed by a user to solve a problem or achieve an objective.
2) A condition or capability that must be met or possessed by a system or system component to satisfy a contract, standard, specification, or other formally imposed documents.
3) A documented representation of a condition or capability as in (1) or (2).

IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications



High-level requirements cascade down to specific details

Source:
https://www.altexsoft.com/blog/business/functional-and-non-functional-requirements-specification-and-types/

Types of Requirements

Business requirements. These include high-level statements of goals, objectives, and needs.

Stakeholder requirements. The needs of discrete stakeholder groups are also specified to define what they expect from a particular solution.

Solution requirements. Solution requirements describe the characteristics that a product must have to meet the needs of the stakeholders and the business itself.

- **Functional requirements** describe how the product must behave, what are its features and functions.
- **Nonfunctional requirements** describe the general characteristics of a system. They are also known as *quality attributes*.

Transition requirements. An additional group of requirements defines what is needed from an organization to successfully move from its current state to its desired state with the new product.

Functional Requirements

Functional requirements are product features or functions that developers must implement to enable users to accomplish their tasks.

So, it’s important to make them clear both for the development team and the stakeholders.

Generally, functional requirements describe system behavior under specific conditions.

For instance:
The search function allows the users to hunt among various invoices if they want to credit an issued invoice.

Functional Requirements

Functional requirements are product features or functions that developers must implement to enable users to accomplish their tasks.

So, it’s important to make them clear both for the development team and the stakeholders.

Generally, functional requirements describe system behavior under specific conditions.

For instance:
The search function allows the users to hunt among various invoices if they want to credit an issued invoice.

Formats and Documents

Requirements are usually written in text, especially for Agile-driven projects.

However, they may also be visuals.

Here are the most common formats and documents:

- Software requirements specification document (SRS)
- Use cases
- User stories
- Work Breakdown Structure (WBS) – functional decomposition
- Prototypes
- Models and diagrams

Software requirements specification document (SRS)

Functional and nonfunctional requirements can be formalized in the requirements specification (SRS) document.

- The SRS contains descriptions of functions and capabilities that the product must provide.
- The document also defines constraints and assumptions.
- The SRS can be a single document communicating functional requirements or it may accompany other software documentation like user stories and use cases.

SRS must include the following sections:

- Purpose. Definitions, system overview, and background.
- Overall description. Assumptions, constraints, business rules, and product vision.
- Specific requirements. System attributes, functional requirements, database requirements, etc.

Example of SRS: *Example of SRS - Web Publishing System - Students' work.pdf*

Use cases

Use cases describe the interaction between the system and external users that leads to achieving particular goals.

Each use case includes three main elements:

- Actors. These are the users outside the system that interact with the system.
- System. The system is described by functional requirements that define an intended behavior of the product.
- Goals. The purposes of the interaction between the users and the system are outlined as goals.

There are two formats to represent use cases:

- Use case specification structured in textual format
- Use case diagram

Use case specification

A use case specification represents the sequence of events along with other information that relates to this use case.

A typical use case specification template includes the following information:

- Description
- Pre- and Post- interaction condition
- Basic interaction path
- Alternative path
- Exception path.

Use case specification

Overview	
Title	[Title of the basic flow use case]
Description	[Short description of the basic flow]
Actors and Interfaces	[Identifies the Actors and Interfaces to components and services that participate in the use case]
Initial Status and Preconditions	[A pre-condition (of a use case) is the state of the system that must be present prior to a use case being performed]
Basic Flow	
STEP 1: … STEP 2: …	
Post Condition	
[A post-condition (of a use case) is a list of possible states the system can be in immediately after a use case has finished]	
Alternative Flow(s)	
[Alternative flows are described here if needed]	

Use case diagram

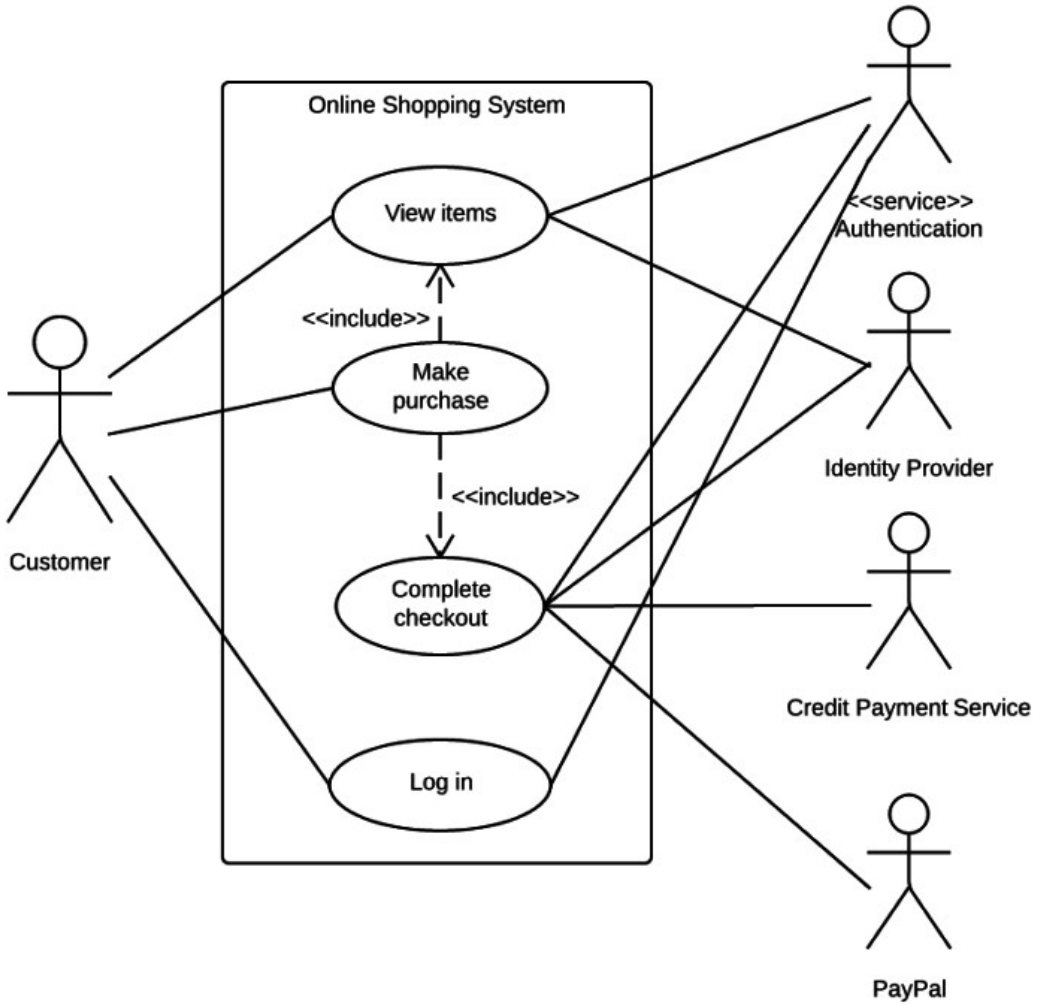
A use case diagram does not contain a lot of details.

It shows a high-level overview of the relationships between actors, different use cases, and the system.

The use case diagram includes the following main elements:

- Use cases. Usually drawn with ovals, use cases represent different use scenarios that actors might have with the system (log in, make a purchase, view items, etc.)
- System boundaries. Boundaries are outlined by the box that groups various use cases in a system.
- Actors. These are the figures that depict external users (people or systems) that interact with the system.
- Associations. Associations are drawn with lines showing different types of relationships between actors and use cases.

Use case diagram



User stories

A user story is a documented description of a software feature seen from the end-user perspective.

The user story describes what exactly the user wants the system to do.

In Agile projects, user stories are organized in a backlog, which is an ordered list of product functions.

Currently, user stories are considered to be the best format for backlog items.

A typical user story is written like this:

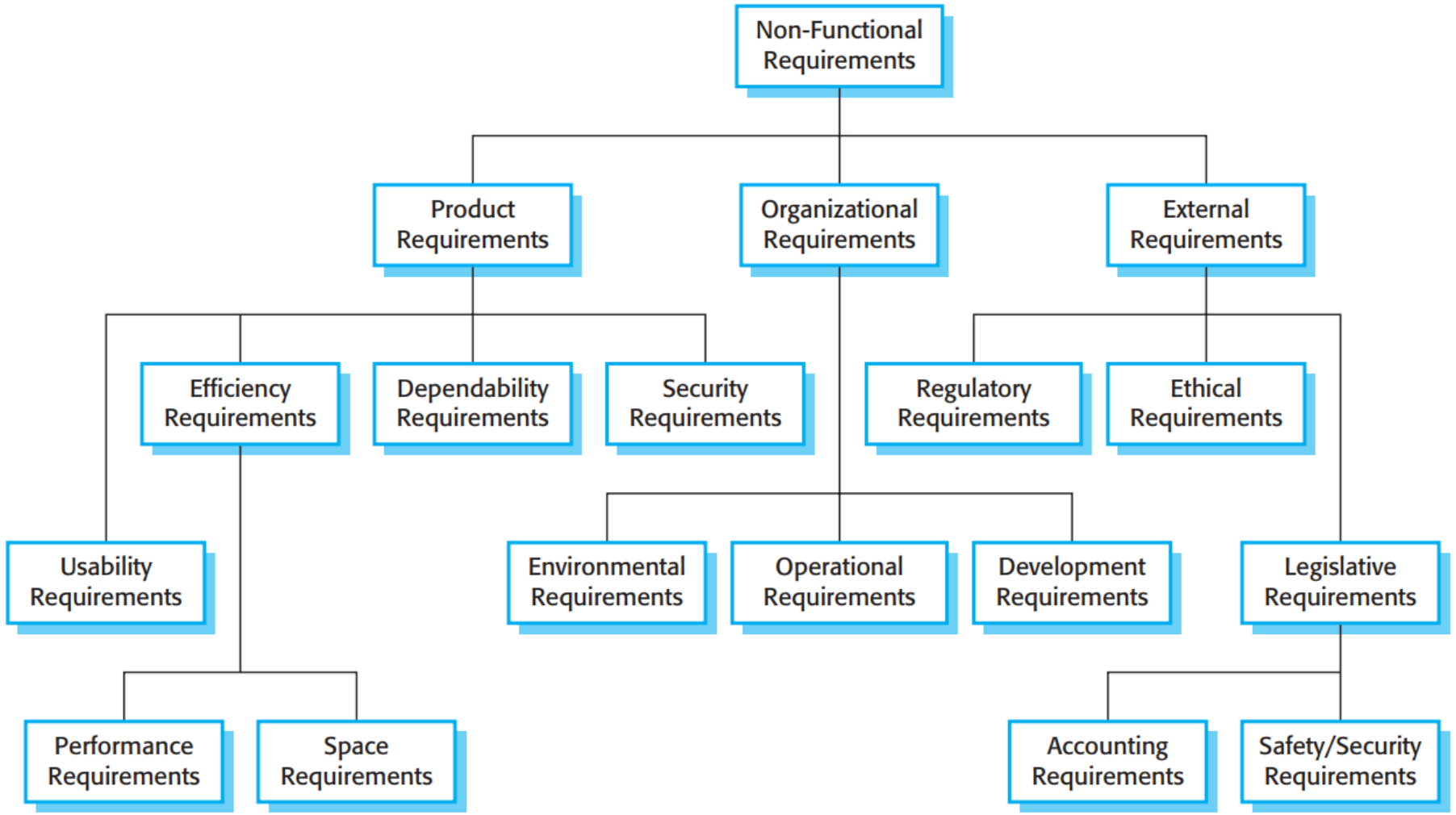
As a <type of user>, I want <some goal> so that <some reason>.

Example:

As an admin, I want to add descriptions to products so that users can later view these descriptions and compare the products.

Nonfunctional requirements

- Nonfunctional requirements describe how a system must behave and establish constraints of its functionality.
- This type of requirements is also known as the system’s quality attributes.
- The non-functional requirements are related to system attributes such as reliability and response time.
- Non-functional requirements arise due to user requirements, budget constraints, organizational policies, and so on.
- These requirements are not related directly to any particular function provided by the system.



Types of nonfunctional requirements

Nonfunctional requirements (1)

- Product requirements: These requirements specify how software product performs. Product requirements comprise the following.
- Efficiency requirements: Describe the extent to which the software makes optimal use of resources, the speed with which the system executes, and the memory it consumes for its operation. For example, the system should be able to operate at least three times faster than the existing system.
- Reliability requirements: Describe the acceptable failure rate of the software. For example, the software should be able to operate even if a hazard occurs.
- Portability requirements: Describe the ease with which the software can be transferred from one platform to another. For example, it should be easy to port the software to a different operating system without the need to redesign the entire software.
- Usability requirements: Describe the ease with which users are able to operate the software. For example, the software should be able to provide access to functionality with fewer keystrokes and mouse clicks.
- Organizational requirements: These requirements are derived from the policies and procedures of an organization.

Nonfunctional requirements (2)

- Delivery requirements: Specify when the software and its documentation are to be delivered to the user.
- Implementation requirements: Describe requirements such as programming language and design method.
- Standards requirements: Describe the process standards to be used during software development. For example, the software should be developed using standards specified by the ISO and IEEE standards.
- External requirements: These requirements include all the requirements that affect the software or its development process externally. External requirements comprise the following.
- Interoperability requirements: Define the way in which different computer based systems will interact with each other in one or more organizations.
- Ethical requirements: Specify the rules and regulations of the software so that they are acceptable to users.
- Legislative requirements: Ensure that the software operates within the legal jurisdiction. For example, pirated software should not be sold.

Nonfunctional requirements (3)

- Security requirements: ensure that the software is protected from unauthorized access to the system and its stored data. It considers different levels of authorization and authentication across different users roles.
- Performance requirements: describe the responsiveness of the system to various user interactions with it. Poor performance leads to negative user experience. It also jeopardizes system safety when it’s is overloaded..
- Availability requirements: Availability is gauged by the period of time that the system’s functionality and services are available for use with all operations. So, scheduled maintenance periods directly influence this parameter. And it’s important to define how the impact of maintenance can be minimized.
- Scalability: Scalability requirements describe how the system can grow without negative influence on its performance. This means serving more users, processing more data, and doing more transactions. Scalability has both hardware and software implications. For instance, you can increase scalability by adding memory, servers, or disk space. On the other hand, you can compress data, use optimizing algorithms, etc.

Characteristics of a Good Requirement (1)

Source: https://www.informit.com/articles/article.aspx?p=1152528&seqNum=4

A requirement needs to meet several criteria to be considered a “good requirement”. Good requirements should have the following characteristics:

- Unambiguous - there should be only one way to interpret the requirement.
- Testable (verifiable) - to be testable, requirements should be clear, precise, and unambiguous.
- Clear (concise, terse, simple, precise) - they should not contain unnecessary verbiage or information; they should be stated clearly and simply.
- Correct - if a requirement contains facts, these facts should be true.
- Understandable - they should be grammatically correct and written in a consistent style; standard conventions should be used.
- Feasible (realistic, possible) - they should be doable within existing constraints such as time, money, and available resources.
- Independent - to understand the requirement, there should not be a need to know any other requirement.

Characteristics of a Good Requirement (2)

- Atomic - the requirement should contain a single traceable element.
- Necessary - the requirement is unnecessary if none of the stakeholders needs the requirement or removing the requirement will not affect the system.
- Implementation-free (abstract) - they should not contain unnecessary design and implementation information.

Besides these criteria for individual requirements, three criteria apply to the set of requirements. The set should be:

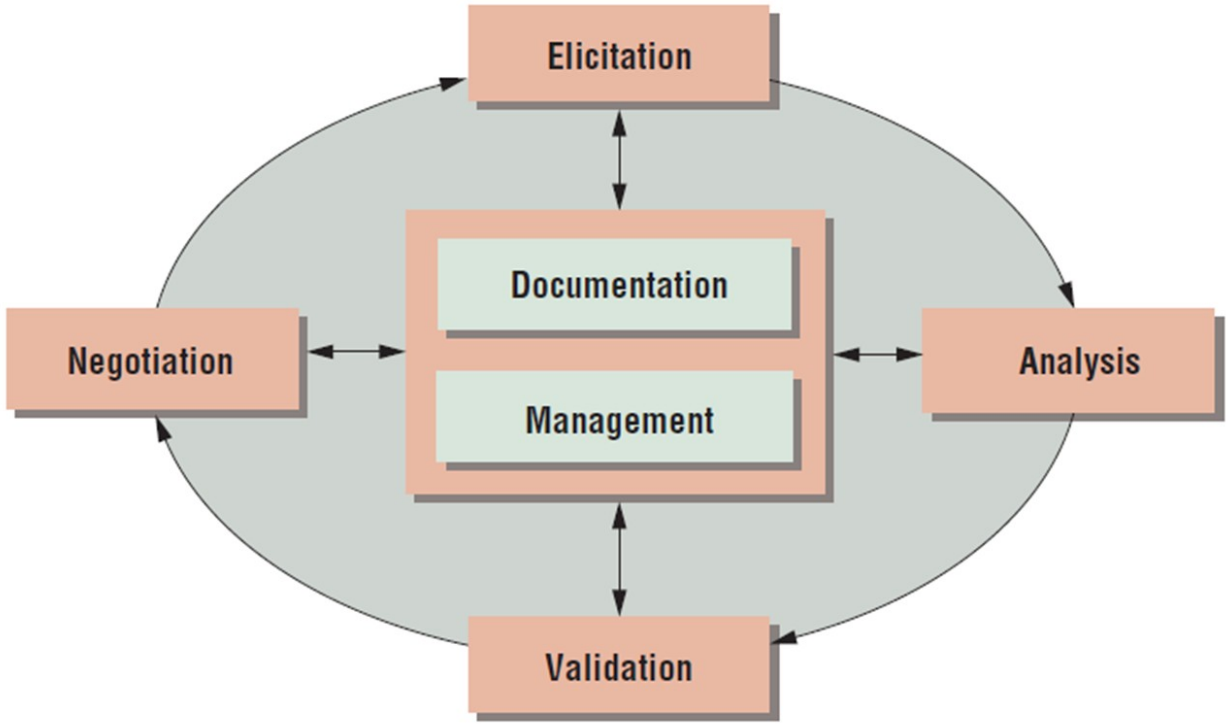
- Consistent - there should not be any conflicts between the requirements; conflicts may be direct or indirect.
- Nonredundant - each requirement should be expressed only once and should not overlap with another requirement.
- Complete - a requirement should be specified for all conditions that can occur. All applicable requirements should be specified.

Characteristics of a Good Requirement (3)

A good requirement should have more criteria. However, they usually can be expressed as a combination of the criteria just discussed:

- Modifiable: If it is atomic and nonredundant, it is usually modifiable.
- Traceable: If it is atomic and has a unique ID, it is usually traceable.

Requirements Engineering Process is a cyclic proces composed of a series of activities that are performed in the requirements phase to express requirements in the Software Requirements Specification (SRS) document.



- **Elicitation**. Identify sources of information about the system and discover the requirements from these.
- **Analysis**. Understand the requirements, their overlaps, and their conflicts.
- **Validation**. Go back to the system stakeholders and check if the requirements are what they really need.
- **Negotiation**. Inevitably, stakeholders' views will differ, and proposed requirements might conflict. Try to reconcile conflicting views and generate a consistent set of requirements.
- **Documentation**. Write down the requirements in a way that stakeholders and software developers can understand.
- **Management**. Control the requirements changes that will inevitably arise.

Requirements traceability matrix

- The requirements traceability matrix is a grid that links product requirements from their origin to the deliverables that satisfy them.
- The implementation of a requirements traceability matrix helps ensure that each requirement adds business value by linking it to the business and project objectives.
- It provides a means to track requirements throughout the project life cycle, helping to ensure that requirements approved in the requirements documentation are delivered at the end of the project.
- Finally, it provides a structure for managing changes to the product scope

Requirements Traceability Matrix

Project Name:	
Cost Center:	
Project Description:	

ID	Associate ID	Requirements Description	Business Needs, Opportunities, Goals, Objectives	Project Objectives	WBS Deliverables	Product Design	Product Development	Test Cases
001	1.0							
	1.1							
	1.2							
	1.2.1							
002	2.0							
	2.1							
	2.1.1							
003	3.0							
	3.1							
	3.2							
004	4.0							
005	5.0							

Example of a Requirements Traceability Matrix taken from the PMBOK Guide, ch. V, page 149

Requirements traceability matrix

- The Requirements Traceability Matrix (RTM) is designed to serve the following purposes:
- To support the capture and management of all categories of requirements and business rules.
- To provide a means of verifying that the project team correctly understands the requirements and business rules.
- To allow for traceability of detailed requirements back to high-level drivers, objectives and benefits and thereby demonstrate to the project sponsor and senior management that all elements of the project can be justified.
- To allow for traceability of the solution design elements back to the requirements to ensure that they have been fully catered for.
- To provide a means of communicating the requirements to the people who will design, implement and test the solution.
- To provide a baseline document to be referred to when determining whether issues with the solution are to be treated as defects or change requests.

Requirements Management Software

Why are requirements management tools important?
Well, in a perfect world:

- The stakeholders on a project would be mind-readers who automatically knew the project’s every specification and assumption
- Every project would benefit from a business analyst well-versed in domain and technical knowledge
- All clients would articulate their expectations with 100% certainty, and
- Every (mammoth) requirements document would be an absolute joy to read.

But we live in the real world.

Source: https://thedigitalprojectmanager.com/requirements-management-tools/

10 Best Requirements Management Tools & Software of 2021
https://thedigitalprojectmanager.com/requirements-management-tools/

This only one example of numerous lists in the Internet.
Be Cautious ! Many different software reviews are available.

- Wrike
- Jama Software
- Orcanos
- IBM Engineering Requirements Management DOORS Next
- Accompa
- Visure Requirements
- Caliber
- ReqSuite
- Pearls
- Perforce Helix RM.

Thank You very much for your attention